

KAPITEL 5: JAVASCRIPT

LERNZIELE

- JavaScript bzgl. Syntax und Grundkonzepten kennen und anwenden können.
- JSON als Notation zur Definition von JavaScript-Objekten anwenden können.
- Das DOM erläutern können.
- Wesentliche Elemente der DOM-API kennen und anwenden können.



5.1 ÜBERBLICK



JavaScript (kurz: JS oder auch ECMA-Script) ist die Programmiersprache des WWW.
Mit JS wird das "Verhalten" einer Webseite realisiert.



Was ist JavaScript genau?

- JavaScript ist eine C-artige **interpretierte** und implizit typisierte Programmiersprache mit objektorientierten Erweiterungen.
- Ab Version 6 gibt es Klassen, jedoch keine Interfaces oder abstrakte Klassen.
- JavaScript wird von modernen Browsern sehr gut unterstützt, es ist keine Zusatz-Software nötig (vgl. Adobe Flash).
- Aus Gründen der Sicherheit wird jedes Script in einer Sandbox ausgeführt, d.h. es ist kein direkter Zugriff auf das Betriebssystem oder das Dateisystem möglich.
- Die Stärke von JavaScript im Browser ergibt sich im Zusammenspiel mit der sogenannten DOM-Schnittstelle für JS. Über sie kann man von JS aus auf die Elemente des aktuellen HTML-Dokuments zugreifen.
- Obige Aussagen beziehen sich auf die im Browser verwendeten JS-Versionen. JS kann z.B. mit nodeJS auch ohne Browser ablaufen.

Wozu wird JavaScript verwendet?

Mit JavaScript lässt sich jeder Aspekt einer HTML-Seite verändern. Diese Vieseitigkeit führt zu folgenden Einsatzbereichen:

- Überprüfung von Eingabedaten (Validierung) und Ausgeben von Meldungen
- Benutzerführung: Menüs, Subdialoge, Registerkarten (tabs), ...
- Beliebige Manipulationen des Inhalts und Layouts einer Webseite, Animation
- Client-seitige Datenspeicherung
- Multimedia: Grafik, Audio, Video
- Senden und Empfangen von Daten ohne Neuladen der Webseite (AJAX)

Zusammenhang zw. HTML, CSS und JS

- **HTML** beschreibt die Struktur
- **CSS** beschreibt das Layout der Elemente
- **JS** beschreibt das (dynamische) Verhalten

GESCHICHTE VON JAVASCRIPT

- 1995 von Netscape eingeführt
- 1998 ECMA-Script, Standardisierung
- Hier behandelt: ECMAscript Version 6 (seit 2015) – u.a. Klassen und Vererbung
- Die Versionshistorie findet man z.B. hier.

TYPESCRIPT (TS)

TypeScript ist eine typisierte Version von JavaScript.

TS-Programme erinnern stark an Java-Quellcode. TS wird mit Hilfe eines „Transpilers“ vor der Ausführung im Browser in JavaScript übersetzt. Mit TS beseitigt man den generellen Nachteil von JavaScript, dass es kaum Fehlerüberprüfung – insbesondere Typprüfungen bei Parametern und Variablen – gibt.

In der Vorlesung wird auf JavaScript verwendet. Weiterführende Werkzeuge (wie Angular) nutzen häufig TypeScript.

ÜBUNG: WAS IST JAVASCRIPT

Was ist JavaScript hauptsächlich?

- [ ] Eine serverseitige Programmiersprache
- [ ] Eine Markup-Sprache für Webseiten
- [ ] Ein Datenbankmanagementsystem
- [ ] Eine clientseitige Skriptsprache für Webbrowser

ÜBUNG: JAVA VS. JAVASCRIPT

Wo liegen die Unterschiede zwischen Java und JavaScript?

- [ ] Java wird in einen Zwischencode compiliert, während JS direkt interpretiert wird.
- [ ] JavaScript-Programme laufen schneller als Java-Programme.
- [ ] Java ist vollständig objektorientiert, JavaScript unterstützt nur Teile des OO-Paradigmas.
- [ ] Java wird hauptsächlich auf Server-, JavaScript hauptsächlich auf Browserseite ausgeführt.

5.2 JAVASCRIPT EINBINDEN

Es gibt vier Möglichkeiten, JavaScript in ein HTML-Dokument einzubinden:

- `<script>`-Tag (an jeder beliebigen Stelle):

```
<script>
  anweisung1;
  anweisung2;
</script>
```

- Im HTML-Code, z.B. als Event-Handler:

```
<input type="button" onclick="alert('Klick!');">
```

- Im HTML-Code als Link

```
<a href="javascript:alert('Klick!');">Test</a>
```

- Einbinden externer JS-Dateien

```
<script src="meinScript1.js"></script>
<script src="meinScript2.js"></script>
```

Bewertung der Möglichkeiten:

- Mit "script" eingebundene JS-Fragmente können nur im aktuellen Dokument genutzt werden, besitzen also eine begrenzte Wiederverwendbarkeit.
- JS-Programmierung "inline" in HTML-Elementen sollte wg. fehlender Wiederverwendbarkeit und schlechtem Debugging vermieden werden. "onclick"-Handler (zur Behandlung von Buttonklicks) beispielsweise sollten aus diesem Grunde idealerweise eine JS-Funktion aufrufen, die in einer eingebunden JS-Datei definiert wird.
- js-Dateien bieten die Vorteile der Wiederverwendbarkeit und der zentralen Wartbarkeit des JS-Codes für eine Webanwendung.

Position der Einbindung

Wo wird JS-Code eingebunden?

- Script-Blöcke und externer JS-Dateien können sowohl im Head- als auch Body-Element eingebunden werden
- Für die Entwicklung von moderner Webseiten ist es wichtig *wo* eingebunden wird
- Wird ein Skript-Element (ohne async/defer) platziert, lädt der Browser an dieser Stelle den Script-Code - je umfangreicher dieser ist, desto länger dauert dies
- Zwingend notwendiger Skript kann bereits im Head-Element geladen werden, i.d.R. (damit Webseiten schnell angezeigt werden können) empfiehlt sich das Laden von Skripten eher am Ende im Body-Element oder die Verwendung des "async" bzw. "defer" Attributs

Hinweis: der Skript muss immer vor der ersten direkten Verwendung geladen werden

Ergänzende Hinweise zum verzögerten Laden von Skripten

Attribut async: Lädt parallel, führt sofort aus (kann HTML-Parsing unterbrechen)

```
<script src="script.js" async></script>
```

Attribut defer: Lädt parallel, führt erst nach HTML-Parsing aus (behält Reihenfolge)

```
<script src="script.js" defer></script>
```

ÜBUNG: EINBINDUNG VON JAVASCRIPT

Welche Aussagen stimmen?

- [ ] JavaScript kann in HTML über Script-Tags und in Event-Attributen eingebettet werden
- [ ] Direkt im Quellcode platzierte JS-Skripte sind zu vermeiden
- [ ] Externe JavaScript-Dateien sind zu vermeiden
- [ ] JavaScript kann nur direkt im HTML-Quellcode platziert werden

ÜBUNG: POSITION DER EINBINDUNG

Wo sollte man den folgenden `script`-Tag für eine externe JavaScript-Datei idealerweise platzieren?

```
<script src="something.js"></script>
```

- [ ] Es spielt keine Rolle, wo der script-Tag platziert wird
- [ ] Direkt nach dem öffnenden body-Tag
- [ ] Im head-Element der HTML-Seite
- [ ] Am Ende des body-Elements der HTML-Seite

ÜBUNG: EINBINDEN EINER SKRIPT-DATEI

Wie bindet man eine externe JavaScript-Datei namens "main.js" korrekt ein?

- ☐ `<javascript file="main.js"></javascript>`
- ☐ `<link rel="javascript" href="main.js">`
- ☐ `<script src="main.js"></script>`
- ☐ `<script href="main.js"></script>`

5.3 SPRACHKONZEPTE



Es gibt sehr gute Tutorials zum Einstieg in JavaScript. Aus diesem Grund lautet die Empfehlung, neben der Beschäftigung mit diesem Skript das W3C-JavaScript-Tutorial durchzuarbeiten und insbesondere die "Try it yourself"-Beispiele auszuprobieren.

Im Folgenden werden wesentliche Aspekte der Sprache kurz angesprochen und am Beispiel erläutert.

Behandelte Themen:

- Datentypen und Konstanten
- Variable
- Kontrollstrukturen
- Arrays
- Objekte
- Klassen
- Funktionen
- Fehlerbehandlung
- JSON (JS Object Notation)

DATENTYPEN UND KONSTANTEN

JavaScript ist implizit typisiert, d.h. der Datentyp einer Variable oder einer Funktion ergibt sich dynamisch durch Wertzuweisung.

JavaScript kennt folgende Datentypen:

- primitive Typen:
  - `number`, z.B. 12 oder 12.345
  - `string`, z.B. "Hallo!" bzw. 'Hallo!'
  - `boolean` mit den Werten `true` bzw. `false`
  - `undefined` mit Wert `undefined` für nicht-zugewiesene Variable
- Der Typ `object`
  - Ein Object ist eine Menge von "properties" bestehend aus Name und Wert.
  - Vordefinierte Konstante: `null` (analog zu Java)
  - Arrays haben den Datentyp `object`
- Der Typ `function` für Funktionen

```
var i = 1;           // number
const f = 1.2;       // number
var b = true;        // boolean
var s = 'Hello';     // string
console.log("Typ(i) = " + typeof (i) + ", Typ(f) = "
    + typeof (f) + ", Typ(b) = " + typeof (b) + ', Typ(s) = '
    + typeof(s));
```

- Mit `var` wird eine Variable deklariert.
- Mit `const` wird eine nach der Initialisierung nicht mehr veränderliche Variable definiert.
- Die Anweisung `console.log(s)` schreibt den String `s` in die JavaScript-Console des Browsers (in der Entwicklungsumgebung des Browsers einzusehen)
- Der Aufruf von `typeof(<variable>)` liefert den Datentyp der Variable als String.
- Der Operator `+` kann analog zu Java für Zahlen und Strings genutzt werden.

ÜBUNG: PRIMITIVE DATENTYPEN

Welche der folgenden sind in JavaScript als primitive Datentypen klassifiziert?

- [ ] Number
- [ ] String
- [ ] Boolean
- [ ] Object
- [ ] Byte
- [ ] Long
- [ ] Undefined

VARIABLE

Eine Variable ist ein Container zum Speichern eines Wertes eines bestimmten Typs.

Variable werden mit dem Schlüsselwort `var` deklariert und erhalten erst durch eine Wertzuweisung einen Datentyp (den des Wertes), z.B.:

```
var i = 5; /* number */
```

Man spricht in JavaScript von *impliziter Typisierung*. Durch weitere Zuweisungen ändert sich der Wert und ggf. auch der Typ.

Mit `let` und `const` wird eine Variable vereinbart, die nur im aktuellen Block gilt (vgl. Block-Scoped), eine mit `var` vereinbarte Variable gilt dagegen für den Rest des Scripts bzw. der aktuellen Funktion (vgl. Function-Scoped oder Global-Scoped). Zusätzlich gilt für `const`, dass Werte nur einmal zugewiesen werden können.

```
if(true) {
    var a = 1;
    let b = 2;
    const c = 2;
}
console.log(a);
console.log(b);
console.log(c);
```

- Funktioniert, da `a` weiterhin gilt wenn mit var definiert wird
- Funktioniert **nicht**, da `b` am Ende des If-Blocks nicht mehr existiert
- Funktioniert **nicht**, da `c` am Ende des If-Blocks nicht mehr existiert

ÜBUNG: VAR, LET UND CONST

Welche Aussagen über `var`, `let` und `const` in JavaScript sind korrekt?

- [ ] `var` ist function-scoped oder global-scoped
- [ ] `const` muss bei der Deklaration initialisiert werden
- [ ] `let` und `const` sind block-scoped
- [ ] Variablen, die mit `let` deklariert wurden, können neu zugewiesen werden
- [ ] Variablen, die mit `var` deklariert wurden, können nach einem Block nicht verwendet werden

OPERATOREN

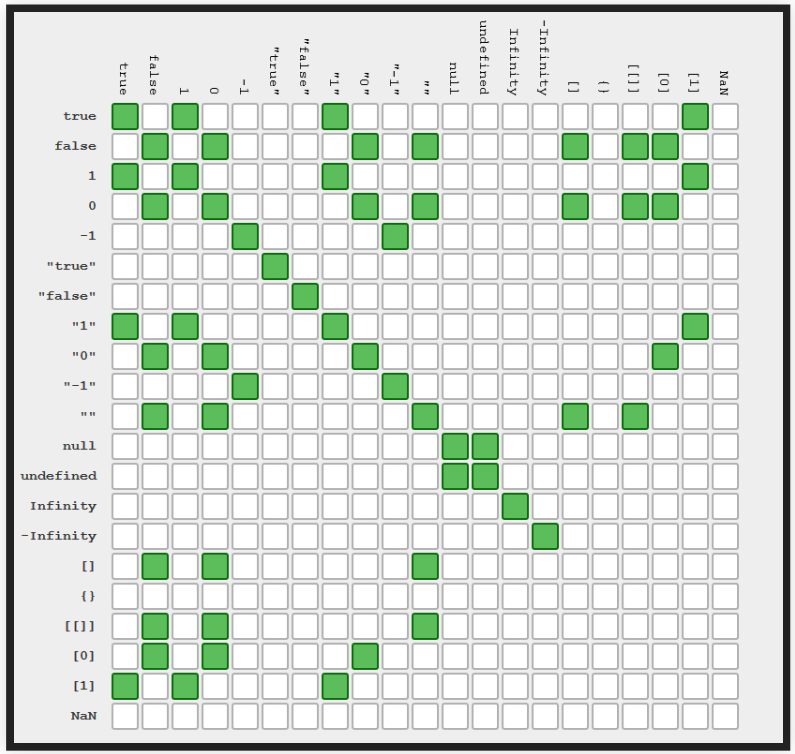
Ebenso stehen die von Java bekannten Standard-Operatoren für Arithmetik und Vergleiche zur Verfügung.

- Arithmetische Operatoren: `+`, `-`, `*`, `/`, `%`, `++`, `--`
- Zuweisungsoperatoren: `=`, `+=`, `-=`, `*=`, `/=`, ...
- Vergleichsoperatoren: `===`, `==`, `<`, `<=`, `>=`, `>`, `!=`, `!==`, ...
- Bitweise Operatoren: `&`, `|`, `^`, `<<`, `>>`, `>>>`, ...

Grundsätzlich versucht der JS-Interpreter, Operanden verschiedener Typen "passend" zu machen. Entsprechend sollte man in der Programmierung sicherstellen, dass die Datentypen kompatibel sind, um überraschende Ergebnisse zu vermeiden.

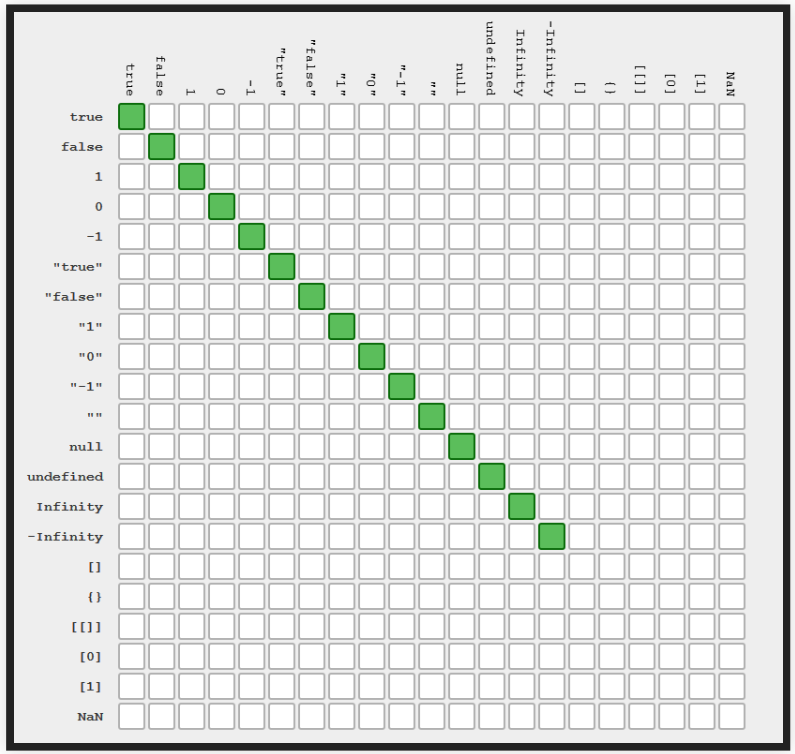
Vergleiche in JavaScript

Bei Verwendung des Vergleichsoperators == kann es in JavaScript zu Typumwandlungen kommen - womit einige Vergleiche ggf. unerwartete Ergebnisse (im Vergleich zu Java) liefern:



Quelle: https://dorey.github.io/JavaScript-Equality-Table/

Alternativer Vergleichsoperator ===



Quelle: https://dorey.github.io/JavaScript-Equality-Table/

ÜBUNG: ADDITION

Was ist das Ergebnis von "5" + 3 in JavaScript?

- [ ] 8 *(Number)*
- [ ] Error
- [ ] 53 *(Number)*
- [ ] "53" *(String)*

ÜBUNG: VERGLEICHSOPERATOR

Recherchieren Sie, was das Ergebnis ist, wenn man in JavaScript den Vergleich `[] == ![]` *verwendet. Ergänzen Sie eine kurze Erklärung, warum dies so sein könnte.*

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

KONTROLLFLUSS

- Es gibt die aus C bzw. Java bekannten Kontrollstrukturen wie : `if...else`, `switch`, `while`, `do...while`, `for`.
- Die Anweisungen können Blöcke mit `{` bzw. `}` nutzen oder einzelne Anweisungen analog zu Java.
- Ebenfalls analog zu Java können `break` und `continue` genutzt werden, benannte Strukturen sind ebenfalls möglich.
- Zusätzlich gibt es eine `for-each`-Schleife zur Iteration über Arrays, z.B. `for(var index in arr)` oder `for(var value of arr)`.

ÜBUNG: KONTROLLSTRUKTUREN

Welche der folgenden Schlüsselworte für Kontrollstrukturen gibt es in JavaScript?

- [ ] if...else
- [ ] goto
- [ ] for...of
- [ ] switch
- [ ] for...in
- [ ] do...while

ÜBUNG: BEDINGUNGEN UND SCHLEIFEN

Was ist das Ergebnis des folgenden Codes?

```
let x = 0;
for (let i = 0; i < 5; i++) {
  if (i === 3) continue;
  x += i;
}
console.log(x);
```

- [ ] 15
- [ ] 10
- [ ] 7
- [ ] 6

ARRAYS

Arrays werden genutzt, um mehrere Werte **beliebigen** Typs in einer einzelnen Variablen zu speichern. Arrays sind Instanzen des Datentyps `object`.

```
var arr0 = [];
var arr1 = [1, 2, 3];
var arr2 = new Array(1, 2, 3);
var arr3 = Array.of(1, 2, 3);
```

- JavaScript erlaubt mehrere Möglichkeiten Arrays zu erzeugen, eine Möglichkeit ist `[]` für ein leeres Array
- Dabei können direkt die initialen Werte angegeben werden
- Daneben gibt es weitere Möglichkeiten



Eine Personenbeschreibung als Array legt man mit Hilfe des Operators "[]" z.B. so an (ein Object ist allerdings besser geeignet, siehe hier):

```
var person = ['Petra', 'Maier', 23];
```

Die Werte eines Arrays werden mit einem ganzzahligen Index (0, 1, ...) angelegt und können über den Index gelesen bzw. verändert werden. Obiges Array könnte man alternativ auch so anlegen:

```
let person = []; // leeres Array anlegen
person[0] = 'Petra';
person[1] = 'Maier';
person[2] = 23;
```

Neue Werte können ans Ende eines Arrays mit `push()` und die Länge lässt sich mit `length` auslesen hinzugefügt werden. Der Variable `person` können wir beispielsweise noch die Angabe des Wohnorts hinzufügen mit:

```
person.push('Ingolstadt');
console.log(person.length);
```

Standard-Sortierung von Arrays

Ein Array aus Zahlen oder Strings kann mit sort() aufsteigend sortiert werden.

```
/* Sortierung eines String-Arrays */
let worte = ['ein', 'bisschen', 'spät', 'ist', 'es'];
worte.sort();
document.writeln('Strings: ' + worte);

/* Sortierung eines Zahlen-Arrays */
document.writeln('<br>');
document.writeln('Zahlen: ' + [33, 1, 100, -100].sort());
```

Dekonstruktion bei Zuweisung

Array-Elemente lassen sich auf Variabeln bei Verwendung des Zuweisungsoperators dekonstruieren.

```
var foo = ["eins", "zwei", "drei"];

var eins = foo[0];
var zwei = foo[1];
var drei = foo[2];

var [eins, zwei, drei] = foo;
```

- Erzeugt ein Array `foo`
- Liest die Werte aus dem Array
- Dekonstruiert das Array in Variablen, `eins`, `zwei` und `drei` enthalten dann die selben Werte wie in `foo[0]`, `foo[1]` und `foo[2]`

ÜBUNG: ARRAYS ERZEUGEN

Welche der folgenden sind gültige Möglichkeiten, ein Array in JavaScript zu erstellen?

- [ ] `let arr3 = Array.of(1, 2, 3);`
- [ ] `let arr4 = (1, 2, 3);`
- [ ] `let arr2 = new Array(1, 2, 3);`
- [ ] `let arr1 = [1, 2, 3];`

ÜBUNG: ARRAY-METHODEN

Arrays sind Objekte in JavaScript, daher können Methoden an diesen Aufgerufen werden. Recherchieren Sie min. zwei Methoden und erklären Sie diese kurz.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

OBJEKTE

JavaScript bietet den Typ `object` zur Definition von Objekten mit Daten und Funktionen. Es ist nicht wie in Java notwendig vorher Klassen zu definieren, aus denen die Objekte erzeugt werden.

Jedes Objekt kann beliebige Eigenschaften ("properties") besitzen, die entweder Daten oder Funktionen (siehe Abschnitt "Funktionen") sein können.

Ein leeres Objekt kann wie folgt angelegt werden:

```
var person1 = {};
var person2 = {name: 'Marie'};
var person3 = new Object();
var person3 = Object();
```



Objekte werden anschließend über den Punkt-Operator (Member-Access-Operator) genutzt. `.` kann nach Erzeugung genutzt werden, um auf bestehende Properties zuzugreifen, aber auch, um neue Properties zu definieren.

```
var hund = {};
hund.alter = 8;
hund.name = 'Woofer';
hund.bellen = function() {
    console.log('wuff');
}
```

Die Verwendung funktioniert dann ebenfalls über den Punkt-Operator, aber auch über die Klammer-Notation wie bei Arrays.

```
console.log(hund.name + ": " + hund.alter);
hund.bellen();
console.log(hund["name"]);
```

Weitere Erläuterungen zum Person-Beispiel:

- Die Funktionen werden als sogenannte "anonyme" Funktionen (diese sind auch vom Typ "function"!) angelegt.
- `this` ist eine vordefiniere Variable und verweist auf das Objekt, dem eine Funktion zugeordnet ist (vgl. Java).

Das Personenobjekt auf der vorherigen Seite lässt sich auch so anlegen:

```
person = {
    vorname: 'Hugo',
    wohnort: '',
    einziehen: function(ort) {
        this.wohnort = ort;
    },
    toString: function() {
        return this.vorname + ' wohnt in ' + this.wohnort;
    }
};
```

Obige Schreibweise des Objektes (`{...}`) ist der später behandelten JavaScript Object Notation (JSON) sehr ähnlich. Der Unterschied besteht darin, dass hier Funktionen definiert werden. Dies ist bei JSON nicht erlaubt. Zusätzlich müssen bei JSON die Namen der Attribute in Anführungszeichen stehen.

Objekte aus Vorlagen erzeugen

```
// Ein Basis-Objekt als "Vorlage"
const person = {
  grüßen() {
    console.log(`Hallo, ich bin ${this.name}`);
  }
};

// Neues Objekt basierend auf person erstellen
const erna = Object.create(person);
erna.name = "Erna";
erna.grüßen(); // "Hallo, ich bin Erna"
```

Mittels `Object.create` kann ein Objekt als Vorlage übergeben werden, die Methode erzeugt dann ein neues Objekt daraus. Wird als Argument `null` übergeben, entsteht ein neues aber leeres Objekt.

Abgrenzung: JavaScript ist eine Prototyping-Sprache

Entgegen der strikten Typisierung in Java und der notwendigen Definition von Klassen können Objekte in JavaScript direkt erzeugt werden. Trotzdem gibt es einen möglichen Zusammenhang für Objekte der selben Art. Dies wird über die versteckte Eigenschaft `[[Prototype]]` (Zugreifbar über `__proto__`) geregelt.

Hiermit wird erneut ein beliebiges Objekt referenziert, jedoch kann das referenzierende Objekt auf die Properties dieses Prototyp-Objects zugreifen und im eigenen Kontext nutzen (vgl. `this`).

```
var p1 = {name: "Eva"};
var p2 = {name: "Max"};
var myProto = {print: function() {console.log(this.name)}}
p1.__proto__ = myProto; // besser Object.setPrototypeOf(p1, myProto);
p2.__proto__ = myProto; // besser Object.setPrototypeOf(p2, myProto);
p1.print();
p2.print();
```

ÜBUNG: OBJEKTE ERZEUGEN

Welche der folgenden sind gültige Möglichkeiten, ein Objekt in JavaScript zu erstellen?

- [ ] `let obj1 = {key: 'value'};`
- [ ] `let obj4 = Object();`
- [ ] `let obj2 = new Object();`
- [ ] `let obj3 = Object.create(null);`

ÜBUNG: ZUGRIFF AUF EIGENSCHAFTEN

Was gibt der folgende Code aus?

```
let person = {name: 'Alice', age: 30, job: undefined};
let key = 'name';
console.log(person.age, person[key], person.job, person.hobby);
```

- [ ] 30, "Alice", undefined, null
- [ ] 30, "Alice", null, undefined
- [ ] 30, "Alice", undefined, undefined
- [ ] 30, undefined, undefined, undefined



FUNKTIONEN

Funktionen werden mit dem Schlüsselwort "function" analog zu Java-Methoden definiert, allerdings ohne Ergebnis- bzw. Parametertypen. Ein Beispiel zum Syntax ist:

```
function appendString(string1, string2, string3) {
    return string1 + ' ' + string2;
}
let s = appendString('Hallo', 'Welt!');
```

- Das Beispiel definiert eine Funktion namens "appendString".
- Der Aufruf wird dann über den Namen zzgl. Klammern realisiert, in den Klammern können die Argumente platziert werden.
- Nicht genutzte Parameter haben den Wert `undefined`, es wird aber kein Fehler erzeugt.



Möglichkeiten zur Definition von Funktionen

```
var add = function(a, b) { return a + b; }
var sub = (a, b) => { return a - b; }
var mul = new Function('a', 'b', 'return a * b;');
function div(a, b) { return a / b; }
```

- Wird kein Name angegeben, wird eine anonyme Funktion erstellt, diese kann z.B. einer Variable zugewiesen werden
- Lambda-Ausdrücke sind ebenfalls möglich und resultieren in Funktionen. Sie werden hier "arrow functions" genannt.
- Über den Aufruf von new Function lassen sich dynamisch Funktionen erstellen, ist jedoch weniger oft notwendig
- Der bereits bekannte Weg über das Schlüsselwort function und einen Namen

Der Datentyp Function

In JavaScript ist es wichtig, dass es den Datentyp Function gibt. Einer Variable oder einem Parameter kann damit immer auch eine Funktion zugewiesen werden.

```
var add = function(a, b) { return a + b; }
function div(a, b) { return a / b; }
var otherAdd = add;
var otherDiv = div;
console.log(otherAdd(1, 2) + ", " + otherDiv(4, 2));
```

Hier werden die Funktionen `add` und `div` den Variablen `otherAdd` und `otherDiv` übergeben. Diese können anschließend ebenfalls aufgerufen werden.

Zu beachten ist, dass bei der Zuweisung die Klammern für die Argumente weggelassen werden müssen.

ÜBUNG: FUNKTIONEN DEFINIEREN

Welche der folgenden sind gültige Möglichkeiten, eine Funktion in JavaScript zu definieren?

- [ ] `var f4 = new Function('a', 'b', 'return a + b')`
- [ ] `function f1() { }`
- [ ] `const f2 = function() { }`
- [ ] `let f3 = () => { }`

ÜBUNG: EIGENSCHAFTEN VON FUNKTIONEN

Welche der folgenden Aussagen über Parameter in JavaScript-Funktionen sind korrekt?

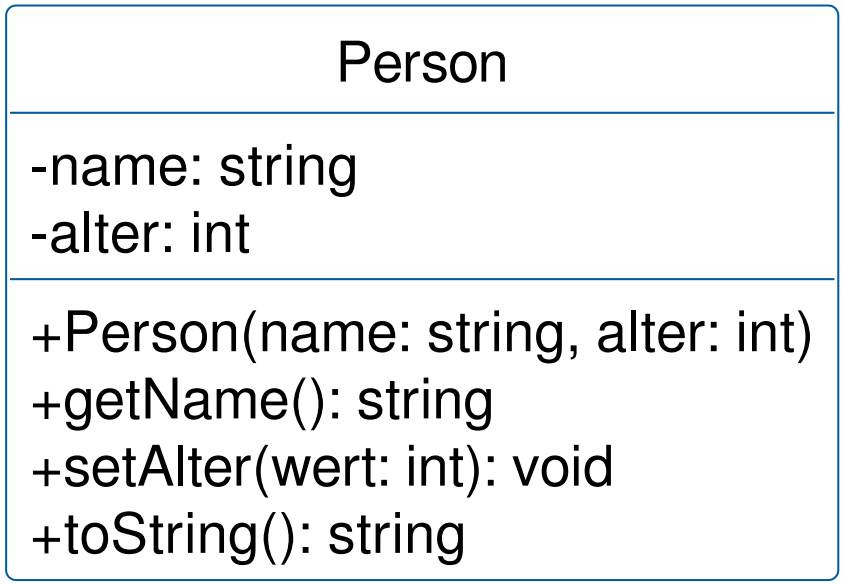
- ☐ Funktionen können eine beliebige Anzahl von Argumenten akzeptieren, unabhängig von der Anzahl der deklarierten Parameter
- ☐ Nicht übergebene Parameter haben den Wert `undefined`
- ☐ Bei der Übergabe einer Funktionen müssen die Klammern `()` hinter dem Namen verwendet werden
- ☐ Funktionen können einer Variable zugewiesen oder als Argument verwendet werden

KLASSEN

Ab Version 6 bietet JavaScript ein Klassenkonzept mit Vererbung an.

Wie man eine Klasse definiert, wird anhand eines Beispiels erläutert.

Wir wollen folgendes Modell umsetzen:



```
class Person {
    constructor(name, alter) {
        this._name = name;
        this._alter = alter;
    }
    get name() { return this._name; }
    set alter(wert) { this._alter = wert; }
    toString() {
        return "name = " + this._name + ", alter = " + this._alter;
    }
}

let anna = new Person('Anna', 23);
console.log(anna.toString());
```

- Wie üblich wird eine Klasse mit `class` definiert.
- Zur Definition des Konstruktors gibt es das Schlüsselwort `constructor`.
- Zugriff auf Eigenschaften mit `this` ist analog zu Java vorhanden.
- Attribute können mit einem "_"-Präfix versehen werden, es gibt keine Sichtbarkeitsstufen wie in Java.
- Getter/Setter werden mit `get`/`set` angelegt und bei Property-Zugriff von außen (z.B. `person.alter = 19;`) automatisch aufgerufen.



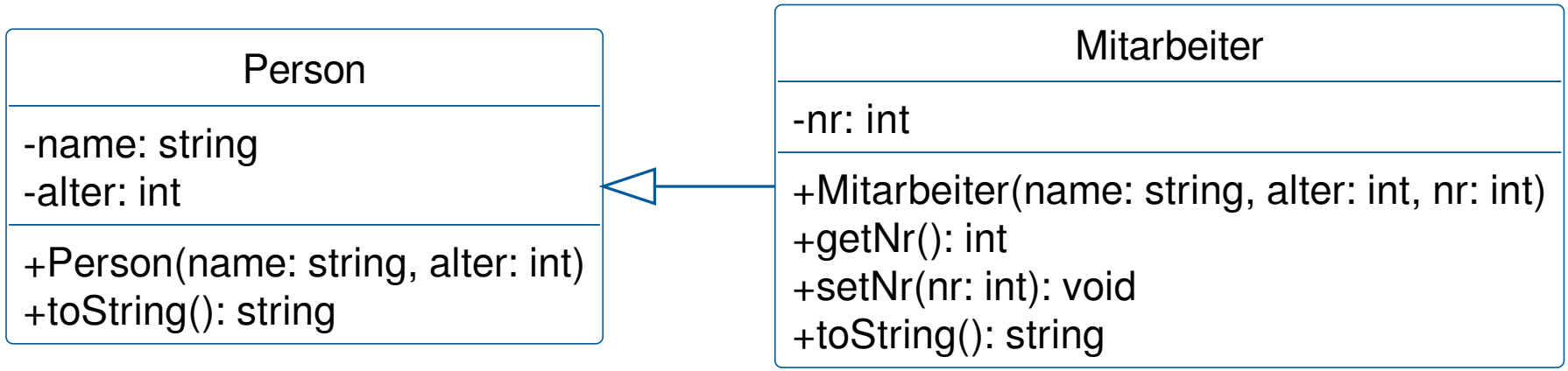
Unterschiede zu Java:

- Attribute werden dynamisch, z.B. im Konstruktor oder in Methoden, angelegt.
- Es gibt keine Möglichkeiten, Sichtbarkeitsstufen zu spezifizieren.
- Getter/Setter werden anders umgesetzt.
- `toString` ist keine geerbete Methode, die man typischerweise überschreibt.
- Properties (get/set), Attribute und Methoden können nicht den selben Namen haben

Vererbung

JavaScript bietet wie Java Einfachvererbung an. Das Vererbungskonzept wird anhand eines kleinen Beispiels veranschaulicht:

- Ein Mitarbeiter ist eine Person mit einer Mitarbeiter-Nummer.
- Die toString-Methode von Mitarbeiter ruft die geerbte toString-Methode von Person auf, um die verfügbare Implementierung für den Personen-Anteil zu nutzen.



Beispiel zur Vererbung: Mitarbeiter erbt von Person

```
class Mitarbeiter extends Person {
    constructor(name, alter, nr) {
        super(name, alter);
        this._nr = nr;
    }
    get nr() { return this._nr; }
    set nr(wert) { this._nr = wert; }
    toString() {
        return super.toString() + ", nr = " + this._nr;
    }
}

let mitarbeiterinAnna = new Mitarbeiter('Anna', 23, 4711);
console.log(mitarbeiterinAnna.toString());
```

- Wie üblich wird eine Ableitung mit "extends" definiert.
- Mit "super" wird der Oberklassenkonstruktor zur Initialisierung der Oberklassenattribute aufgerufen.
- Wie in Java wird "super" auch zum Aufruf von gleichnamigen Methoden der Oberklasse genutzt.

FEHLERBEHANDLUNG

JavaScript bietet eine vereinfachte Version des von Java her bekannten Exception-Konzepts.

- try-catch-Blöcke und throw-Anweisungen sind analog nutzbar.
- Anstelle eines Exception-Objekts wird ein String verwendet, der die Fehlerbeschreibung enthält.

Anmerkung: Class Error mit eigenen Ableitungen nutzbar (analog zu Java), siehe z.B. bei developer.mozilla.org

Beispiel: Klasse Person – Setter für "alter"-Property mit Wertüberprüfung

```
set alter(wert) {
    if (wert < 0 || wert > 120) {                    // Prüfung
        throw 'Ungültiger Wert für Alter: ' + wert; // Exception werfen
    }
    this._alter = wert;
}
```

Anwendung: Fehlerbehandlung im try-catch-Block

```
try {
    let paula = new Mitarbeiter('Paula', 23, 1);
    paula.alter = -23;  // -> wirft Exception
    document.writeln('Paula: ' + paula.toString());
} catch (e) {           // -> Catch-Block wird ausgeführt
    document.writeln('Fehler beim Setzen des Alters: ' + e + '<br>');
}
```



5.4 JSON

JavaScript bietet eine sogenannte Objekt-Literal Notation zur Erzeugung von Objekten. Ein Beispiel in JavaScript wäre:

```
var person = {
    name: "Peter Maier",
    alter: 25
};
```

Im Bereich der Webtechnologien wird ein ähnlicher Syntax zum Transport von Daten verwendet, die sogenannte JavaScript Object Notation. Diese lehnt sich an die Object-Literal Notation an. Ein JSON-Beispiel wäre:

```
{
    "name": "Peter Maier",
    "alter": 25
}
```

Zu beachten ist, dass zum Beispiel die Schlüssel immer in Anführungszeichen stehen müssen und der Umfang an Datentypen beschränkt ist.

Die JSON-Syntax erlaubt die Definition eines Objekts als Folge von Paaren aus einem Property-Namen (Typ String) und einem Wert.

Erlaubte Wert-Typen sind: String, Zahl, Boolean (true/false), null, Object, Array.

Die Syntax findet man z.B. hier.

Beachte: Im Gegensatz zu JavaScript erlaubt JSON nicht, Funktionen für die Datenobjekte zu definieren.

Wie Objekte können ebenfalls Arrays definiert werden. Gültige JSON-Snippets sind die folgenden:

```
[1, 2, 3, 4]
```

```
[{"name": "Max"}, {"name": "Eva"}]
```

```
{"list": [1, 2, 3, 4]}
```

JSON wird häufig genutzt, um Daten zu speichern oder über das WWW zu senden, da die Verarbeitung mit JS einfach und das Format leicht zu lesen ist:

```
// Speichern eines Person-Objekts in Browser Local Storage
const personAlsString = JSON.stringify({
    name: "Peter Maier",
    alter: 25
});
localStorage.person = personAlsString;
// Laden der Person aus Local Storage
const personGeladen = JSON.parse(localStorage.person);
```

- Zuerst wird das JavaScript-Objekt in ein JSON-Object mit "stringify" umgewandelt
- Jeder Browser unterstützt einen key/value-Speicher namens "Local Storage"
- Der String wird unter dem Schlüssel "person" im Local Storage gespeichert.
- Mithilfe von "parse" wird der aus dem Local Storage geladene String wieder in ein JSON-Objekt umgewandelt.

ÜBUNG: JSON

Was bedeutet die Abkürzung JSON?

- [ ] Java Standard Object Nomenclature
- [ ] Java Syntax Object Network
- [ ] JavaScript Object Notation
- [ ] JavaScript Operational Nodes

ÜBUNG: JSON-OBJEKTE

Welches Zeichen wird verwendet, um ein JSON-Objekt zu umschließen?

- [ ] Beispiel 4: ()
- [ ] Beispiel 2: {}
- [ ] Beispiel 1: []
- [ ] Beispiel 3: <>

ÜBUNG: POSITIONEN IN JSON-OBJEKTEN

Gegeben ist eine Liste von 2D Positionen x/y: `4/1`, `10/10`, `2/8`. Welche folgende Variante ist eine passende Repräsentation in JSON.

- ☐ Variante A: `{[4, 1], [10, 10], [2, 8]}`
- ☐ Variante C: `[4/1, 10/10, 2/8]`
- ☐ Variante B: `[{x: 4, y: 1}, {x: 10, y: 10}, {x: 2, y: 8}]`



5.5 DOM

Was ist das Document Object Model (DOM)?



Das DOM ist eine Programmierschnittstelle (API) zur Manipulation von HTML-Dokumenten.

Das DOM repräsentiert ein HTML-Dokument als Baumstruktur bestehend aus Knoten-Objekten (nodes).

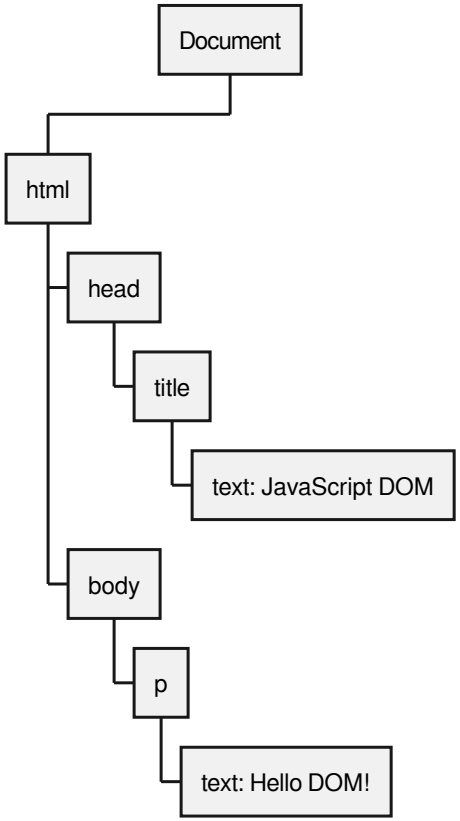
Es bietet Methoden zum Hinzufügen und Entfernen von Nodes sowie zum Modifizieren bestehender Nodes.

BAUMDARSTELLUNG

Wir betrachten folgendes HTML-Dokument:

```
<html>
    <head>
        <title>JavaScript DOM</title>
    </head>
    <body>
        <p>Hello DOM!</p>
    </body>
</html>
```

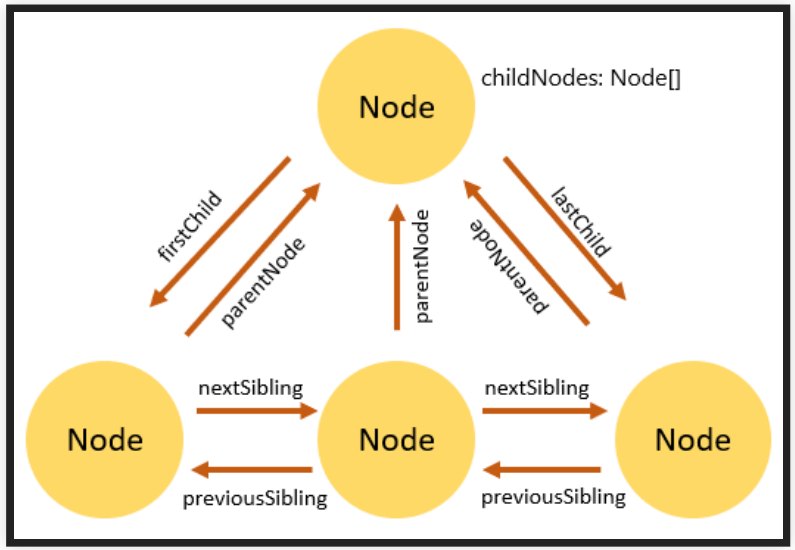
Der folgende Baum repräsentiert das HTML-Dokument als Baumstruktur:



- Der "Document"-Knoten ist die Wurzel des Baumes. Diese besitzt mit dem html-Knoten einen Kindknoten. In JS ist die Wurzel über die Variable "document" verfügbar.
- Analog werden alle Tags des gegebenen HTML-Codes als Knoten im Baum repräsentiert.

BEZIEHUNGEN ZWISCHEN KNOTEN

Jeder Knoten steht in Beziehungen zu den anderen Knoten der Baumstruktur. Über entsprechende Methoden und Attribute des Node-Interface kann man sich durch den Baum bewegen:



BAUMELEMENTE FINDEN

Elemente im Baum können mittels verschieden Funktionen der DOM-API gefunden werden:

- `getElementById`: Finde ein Element mittels Id-Attribut
- `getElementsByName`: Finde ein oder mehrere Elemente mittels Name-Attribut
- `getElementsByTagName`: Finde ein oder mehrere Elemente mittels Tag-Name
- `getElementsByClassName`: Finde ein oder mehrere Elemente mittels CSS-Class-Name
- `querySelector`: Finde ein Elemente mittels CSS-Selector
- `querySelectorAll`: Finde ein oder mehrere Elemente mittels CSS-Selector

ÜBUNG: EIN ELEMENT SUCHEN

Welche der folgenden Methoden können verwendet werden, um ein einzelnes Element im DOM zu finden?

- [ ] querySelectorAll()
- [ ] getElementById()
- [ ] getElementsByClassName()
- [ ] getElementsByTagName()
- [ ] querySelector()

ÜBUNG: MEHRERE ELEMENTE SUCHEN

Welche Methode würden Sie verwenden, um alle Elemente mit der Klasse "highlight" zu finden?

- [ ] `document.querySelector('.highlight')`
- [ ] `document.getElementByClassName('highlight')`
- [ ] `document.querySelectorAll('.highlight')`
- [ ] `document.getElementsByClassName('highlight')`

TRAVERSIEREN DURCH DEN BAUM

Mittels verschiedener Attribute kann durch den Baum navigiert werden

- `parentNode` / `parentElement`: Liefert den Vater
- `children` und `childNodes`: Liefert alle Kinder-Elemente
- `childNodes`: Liefert jedes Kind, auch Text-Knoten
- `firstChild` / `lastChild`: Liefert das erste / letzte Kind
- `nextElementSibling`: Liefert das nächste Nachbar-Element
- `nextSibling`: Liefert den nächsten Nachbar-Knoten, könnte auch einen Text-Knoten liefern
- `previousElementSibling`: Liefert das vorherige Nachbar-Element
- `previousSibling`: Liefert den vorherigen Nachbar-Knoten, könnte auch einen Text-Knoten liefern

ÜBUNG: ATTRIBUTE ZUM TRAVERSIEREN

*Welche der folgenden Gruppen enthält **ausschließlich** korrekte Eigenschaften zum Traversieren des DOM?*

- [ ] superiorNode, nextNode, previousElement, offspring
- [ ] parentElement, firstChild, lastElementChild, childNodes
- [ ] ancestorElement, followingSibling, precedingSibling, descendantNodes
- [ ] parentNode, nextSibling, previousElementSibling, firstChild
- [ ] parentNode, nextElementSibling, previousSibling, children
- [ ] parentElement, firstElementChild, lastChild, childNodes

MANIPULATION DES DOM-BAUMES

Mit bestimmten Methoden an einem DOM-Element lässt sich der Baum verändern:

- `appendChild(node)`: Fügt ein Element als Kind an
- `removeChild(node)`: Entfernt ein Kind
- `insertBefore(node, ref)`: Fügt ein neues Kind bevor ein existierendes
- `createElement(tagName)`: Erzeugt ein neues Element
- `setAttribute(name, value)`: Setzt den Wert eines Attributes
- `getAttribute(name)`: Liefert den Wert eines Attributes

ÜBUNG: ELEMENT ERZEUGEN

Welche Methode wird verwendet, um ein neues Element zu erstellen?

- ☐ `document.makeElement()`
- ☐ `document.newElement()`
- ☐ `document.createElement()`
- ☐ `document.generateElement()`

CSS-ATTRIBUTE SETZEN

Es lassen sich auch CSS-Attribute direkt modifizieren

- Über das Attribut `style` kann auf CSS-Attribute zugegriffen werden, die direkt am Element definiert sind:
  - Beispiel: `document.getElementById('test').style.color` liefert im Fall von `<div id="test" style="color: red">` den Wert "rot"
  - Die CSS-Attribute werden in Camel-Case und ohne Bindestriche formuliert
- Über das Attribut `className` kann auf die CSS-Klassen eines Elements zugegriffen werden
  - Beispiel: `document.getElementById('test').className` liefert im Fall von `<div id="test" class="foo">` den Wert "foo"
- Über das Attribut `classList` kann auf die CSS-Klassen als Liste mit Operationen wie `add` und `remove` zugegriffen werden

ÜBUNG: CSS KLASSE HINZUFÜGEN

Wie fügt man eine neue Klasse zu einem Element hinzu, ohne bestehende Klassen zu überschreiben?

- [ ] element.class += " newClass"
- [ ] element.className += " newClass"
- [ ] element.addClassName("newClass")
- [ ] element.classList.add("newClass")

EREIGNISSE BEHANDELN

JavaScript unterstützt Events rund um die Interaktion mit einem HTML-Dokument.
Dies umfasst zum Beispiel:

- `onload`: Wenn das Dokument geladen wurde, z.B. am Body-Tag platziert
- `onclick`: Wenn das Element angeklickt wurde
- `onmouseenter`: Wenn die Maus über dem Element ist
- `onfocus`: Wenn ein Eingabefeld fokusiert wird

Neben der HTML-basierten Definition gibt es Möglichkeiten in JavaScript:

```
let foo = document.getElementById('test');
foo.addEventListener('click', function(event) {
    console.log(event);
});
```

Mithilfe der Methode `addEventListener` wird an einem Element ein Listener registriert, in diesem Fall für das Ereignis `click`.



5.6 ÜBUNGEN

